

1. Basic data types in Python with examples.

- Python has several built-in data types:
 - int** – integers like `x = 10`.
 - float** – real numbers with decimals, e.g., `y = 3.14`.
 - bool** – Boolean values `True` or `False`.
 - str** – sequence of characters, e.g., `'Hello'`.
 - list, tuple, set, dict** – collection types for grouping data.
- These types are automatically assigned based on the value given to a variable.
- Dynamic typing** allows the type to change during execution.

2. String operations in Python with examples.

- Strings support multiple operations like:
 - Concatenation (+)** → `'Hi' + 'There' → 'HiThere'`.
 - Repetition (*)** → `'Hi' * 3 → 'HiHiHi'`.
 - Slicing** → `'Hello'[1:4] → 'ello'`.
 - Methods** → `.upper()`, `.lower()`, `.replace()`, `.strip()`.
 - Comparison** → `'apple' < 'banana' → True`.
- Strings are immutable**; every modification creates a new string.

3. Difference between mutable and immutable types in Python.

- Mutable objects** can change after creation — e.g., *list, dict, set*.
- Immutable objects** cannot change — e.g., *int, str, tuple*.
- Example:

```
lst = [1, 2]
lst.append(3) # mutable
s = "Hi"
s += "!" # new string created (immutable)
```

- Mutability affects performance and behavior in functions.

4. Procedural programming with an example Python program.

- Procedural programming follows a **top-down** approach using **functions**.
- It divides tasks into reusable procedures.
- Example:

```
def add(a, b):
    return a + b
print(add(2, 3))
```

- Promotes **code reusability** and **structured design**.
- Flow control is handled using loops, conditions, and functions.

5. Control flow statements in Python with examples.

- Control flow determines the sequence of execution:
 - Sequential** – executes line by line.
 - Selection** – `if`, `elif`, `else` for decision-making.
 - Iteration** – `for`, `while` for repetition.
- Example:

```
for i in range(3):
    print(i)
```

- These structures manage program logic and repetition effectively.

6. Loops in Python with syntax and examples.

- Loops **repeat** a block of code multiple times.
- for loop** – iterates over a sequence:

```
for i in range(5):
    print(i)
```

- while loop** – repeats while a condition is true:

```
i = 0
while i < 5:
    print(i)
    i += 1
```

- break** and **continue** alter loop flow.
- Loops help automate repetitive tasks.

7. Standard library modules.

- math** – Mathematical functions: `math.sqrt(16) → 4.0`.
- random** – Generates random values: `random.randint(1, 10)`.
- os** – Interacts with the operating system: `os.getcwd()`.
- These built-in modules simplify tasks through pre-defined functionality.

8. Exception Handling in Python with examples.

- Exception handling prevents program crashes from runtime errors.
- Syntax:

```
try:
    a = 10 / 0
except ZeroDivisionError:
    print("Cannot divide by zero")
finally:
    print("Done")
```

- try** checks for errors, **except** handles them, and **finally** executes cleanup.
- Improves **reliability** and **user experience**.

9. OOP concepts in Python with examples.

- Object-Oriented Programming (OOP) focuses on **classes** and **objects**.
- Key principles:
 - Encapsulation**: data hiding.
 - Inheritance**: code reuse.
 - Polymorphism**: method overriding.
- Example:

```
class A:
    def show(self): print("A")
class B(A):
    def show(self): print("B")
B().show()
```

- Encourages **modular**, **reusable**, and **maintainable** code.

10. File read and write operations with examples.

- File handling allows **data storage and retrieval**.
- Example:

```
f = open("data.txt", "w")
f.write("Hello")
f.close()
```

```
f = open("data.txt", "r")
print(f.read())
```

- Modes: `'r'`, `'w'`, `'a'`, `'b'`.
- Always **close files** or use `with open()` for safety.

11. Inheritance with a Python program.

1. Inheritance allows a class to **acquire properties** of another class.
2. Example:

```
class Parent:
    def greet(self): print("Hello")
class Child(Parent):
    def greet(self): print("Hi Child")
Child().greet()
```

3. Promotes **code reuse** and **polymorphism** by extending base class functionality.

12. Custom modules

1. A **module** is a .py file containing reusable code.
2. Example:
 - mymod.py → def greet(): print("Hi").
 - Usage:

```
import mymod
mymod.greet()
```

3. Modules **organize large programs** and support **code sharing**.

13. Functions in Python with examples.

1. A function is a **reusable block of code** that performs a specific task.
2. Syntax:

```
def greet(name):
    return "Hello " + name
print(greet("Vijay"))
```

3. Functions may take **parameters** and **return values**.
4. Support default, keyword, and variable-length arguments.
5. Encourage **code reuse** and **better organization**.

14. Difference between local and global variables in Python

1. **Local variables:** declared inside a function, accessible only there.
2. **Global variables:** defined outside functions, accessible anywhere.
3. Example:

```
x = 10
def show():
    y = 5
    print(x, y)
show()
```

4. The keyword **global** allows modifying a global variable inside a function.
5. Proper scope management prevents unintended changes.

15. Regular Expressions (re module) in Python.

1. The **re module** is used for **pattern matching** and **text processing**.
2. Common functions:
 - re.search() → finds a match.
 - re.findall() → returns all matches.
 - re.sub() → replaces text.
3. Example:

```
import re
print(re.findall(r'\d+', 'a1b2c3'))
```

4. Useful for validation (email, phone numbers) and text cleaning.

16. Custom Exception handling.

1. Python allows creating **custom exceptions** for specific cases.
2. Example:

```
class InvalidAge(Exception): pass
age = int(input())
if age < 18:
    raise InvalidAge("Not eligible")
```

3. **raise** triggers the exception, **try-except** handles it.
4. Improves clarity and control in large applications.

17. File handling using binary files in Python.

1. **Binary files** store data as bytes, not text.
2. Used for **images**, **videos**, or serialized data.
3. Example:

```
with open("data.bin", "wb") as f:
    f.write(b"Python")
with open("data.bin", "rb") as f:
    print(f.read())
```

4. Modes: 'rb', 'wb', 'ab'.
5. Ensures exact byte-level storage without encoding issues.